

Arquitectura Software - Laboratorio 01

Augusto Pedicino Florez

Erick Salazar Suarez

Felipe Garrido Flores

Pontificia Universidad Javeriana

2026-05-09

Marco conceptual

El desarrollo de aplicaciones modernas requiere el uso de herramientas y tecnologías que permitan construir sistemas organizados, mantenibles y conectados con bases de datos. En este laboratorio se implementa una aplicación web utilizando el ecosistema de Microsoft basado en .NET, SQL Server y Entity Framework Core.

1.1. Tecnologías

1.1.1. ASP.NET Core

ASP.NET Core es un framework de desarrollo de aplicaciones web multiplataforma desarrollado por Microsoft. Permite construir aplicaciones web, APIs y servicios utilizando el lenguaje C# y el framework .NET.

Entre las principales características de ASP.NET Core se encuentran:

- Desarrollo de aplicaciones web y APIs REST.
- Integración con bases de datos mediante Entity Framework Core.
- Compatibilidad multiplataforma.
- Manejo de rutas, controladores y vistas.

1.1.2. Motor de plantillas Razor

Razor es el motor de plantillas (template engine) que usa ASP.NET para generar HTML dinámico desde código C#. La idea principal de Razor es que en un mismo archivo **.cshtml** se pueda mezclar:

- HTML
- código C#
- datos del servidor

Un ejemplo de esto es:

```
<h1>Hola @Model.Nombre</h1>
```

```
@if(Model.Edad >= 18) {  
    <p>Es mayor de edad</p>  
}
```

1.1.3. Entity Framework Core

Entity Framework Core es un ORM (Object Relational Mapper) para .NET. Un ORM permite mapear tablas de bases de datos a clases de programación, facilitando la interacción entre la aplicación y la base de datos. En lugar de escribir consultas SQL manualmente para todas las operaciones, Entity Framework Core permite trabajar utilizando objetos y clases en C#. Las entidades representan las tablas de la base de datos, mientras que el contexto administra la conexión y las operaciones realizadas sobre dichas entidades.

1.1.4. Visual Studio Community 2022

Visual Studio Community 2022 es el entorno de desarrollo (IDE) utilizado para construir la aplicación.

El IDE proporciona herramientas para:

- Edición de código.
- Compilación.
- Depuración.
- Administración de paquetes.
- Integración con Git.
- Diseño y ejecución de proyectos ASP.NET Core.

También permite administrar dependencias mediante **NuGet**.

1.1.5. NuGet

NuGet es el administrador de paquetes utilizado en el ecosistema .NET. Permite instalar librerías externas necesarias para el desarrollo de aplicaciones.

Se listan los principales paquetes que se utilizarán para el **laboratorio**:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools

Estos paquetes permiten la integración entre ASP.NET Core y SQL Server mediante **Entity Framework Core**.

1.1.6. SQL Server Express

Microsoft SQL Server Express es un sistema de gestión de bases de datos relacional (RDBMS) desarrollado por Microsoft. Se utiliza para almacenar y administrar la información persistente de la aplicación. Las bases de datos relacionales organizan la información en tablas compuestas por filas y columnas, permitiendo establecer relaciones entre diferentes conjuntos de datos mediante claves primarias y foráneas.

1.1.7. SQL Server Management Studio (SSMS)

SQL Server Management Studio es una herramienta gráfica utilizada para administrar servidores SQL Server. Permite:

- Crear bases de datos.
- Ejecutar scripts SQL.
- Consultar información.
- Administrar usuarios y permisos.
- Diseñar tablas y relaciones.

1.1.8. DDL y DML

En bases de datos relacionales, el lenguaje SQL se divide en diferentes categorías según el tipo de operación realizada.

1.1.8.1. DDL (Data Definition Language)

El DDL corresponde a las instrucciones encargadas de definir la estructura de la base de datos. Algunas operaciones comunes son:

- CREATE
- ALTER
- DROP

Estas instrucciones permiten crear tablas, modificar estructuras y eliminar objetos de la base de datos.

1.1.8.2. DML (Data Manipulation Language)

El DML corresponde a las instrucciones utilizadas para manipular la información almacenada. Algunas operaciones comunes son:

- INSERT
- UPDATE
- DELETE
- SELECT

Estas instrucciones permiten insertar, consultar y modificar datos dentro de las tablas.

1.2. Patrón Arquitectónico

1.2.1. Arquitectura MVC (Modelo-Vista-Controlador)

El laboratorio utiliza el patrón arquitectónico MVC (Model-View-Controller), ampliamente utilizado en aplicaciones web.

Este patrón divide la aplicación en tres componentes principales:

1.2.1.1. Modelo (Model)

El modelo representa la lógica de negocio y el acceso a los datos. El modelo se encarga de interactuar con la base de datos y representar la información del sistema.

Para el laboratorio se manejaron:

- Entidades generadas con Entity Framework Core.
- Contexto de base de datos.
- Repositorios.
- Interfaces.

1.2.1.2. Vista (View)

La vista corresponde a la interfaz de usuario presentada al usuario final. Su función es mostrar la información proveniente del modelo de manera organizada.

En aplicaciones ASP.NET Core MVC, las vistas suelen desarrollarse utilizando **Razor**.

1.2.1.3. Controlador (Controller)

El controlador actúa como intermediario entre las vistas y el modelo. El controlador coordina el flujo de la aplicación.

Sus funciones principales son:

- Recibir solicitudes HTTP.
- Procesar peticiones.
- Invocar lógica de negocio.
- Retornar respuestas o vistas.

1.2.2. Patrón Repositorio

El laboratorio también utiliza el patrón repositorio, el cual abstrae el acceso a los datos mediante clases especializadas.

El objetivo principal es desacoplar la lógica de negocio de las operaciones de persistencia.

Entre sus ventajas se encuentran:

Mayor organización del código. Reutilización de lógica. Facilidad de mantenimiento. Separación de responsabilidades.

1.2.3. Arquitectura por capas

La aplicación desarrollada utilizando el patrón **MVC** puede entenderse también como un subconjunto o **caso particular** de la **arquitectura por capas**, donde cada componente cumple responsabilidades específicas y se genera comunicación entre capas adyacentes.

Extrapolando a una arquitectura de capas para el caso específico del laboratorio, se tiene lo siguiente:

- **Capa de presentación:** Encargada de las vistas y controladores.
- **Capa lógica:** Contiene reglas de negocio y procesamiento.
- **Capa de acceso a datos:** Encargada de la interacción con *SQL Server* mediante *Entity Framework Core*.
- **Capa de persistencia:** Corresponde a la base de datos *persona_db*.

La separación en capas facilita el mantenimiento y evolución del sistema

Diseño

2.1. Arquitectura

2.2. C4 model

2.2.1. Diagrama de contexto

2.2.2. Diagrama de contenedores

2.2.3. Diagrama de componentes

2.3. Decisiones de diseño

Procedimiento

Conclusiones y lecciones aprendidas

Referencias